

=====

CTE

====

common table expression

=> simplifying the complex queries

=> improves the readability of code

-- finding avg order items per order and avg amount per order

```
select avg(num_order_items) as avg_order_items,  
avg(order_amount) as avg_order_amount  
from (select order_item_order_id as order_id,  
count(*) as num_order_items,  
sum(order_item_subtotal) as order_amount  
from order_items  
group by order_item_order_id) x;
```

```
with aggregated_results as (select order_item_order_id as order_id,  
count(*) as num_order_items,  
sum(order_item_subtotal) as order_amount  
from order_items  
group by order_item_order_id)  
select avg(num_order_items) as avg_order_items,  
avg(order_amount) as avg_order_amount  
from aggregated_results;
```

subqueries

=====

query3(query2(query1))

ctes

=====

query1

query2

query3

-- big orders

```
select * from  
(select order_item_order_id as order_id,  
count(*) as num_order_items,  
sum(order_item_subtotal) as order_amount  
from order_items  
group by order_item_order_id) a  
join  
(select avg(num_order_items) as avg_order_items,  
avg(order_amount) as avg_order_amount
```

```

from (select order_item_order_id as order_id,
count(*) as num_order_items,
sum(order_item_subtotal) as order_amount
from order_items
group by order_item_order_id) x) b
on num_order_items > avg_order_items
and order_amount > avg_order_amount;

```

```

with grouped as (select order_item_order_id as order_id,
count(*) as num_order_items,
sum(order_item_subtotal) as order_amount
from order_items
group by order_item_order_id),
averaged as (select avg(num_order_items) as avg_order_items,
avg(order_amount) as avg_order_amount
from grouped)
select * from grouped join averaged
on num_order_items > avg_order_items
and order_amount > avg_order_amount;

```

1. group the order_id
2. avg of number of order items and order amount
3. join the 2 tables and have the filter condition

-- for finding big orders

```

select order_item_order_id as order_id,
count(*) as num_order_items,
sum(order_item_subtotal) as order_amount
from order_items
group by order_item_order_id
having count(*) > (select avg(num_order_items) as avg_order_items
from (select order_item_order_id as order_id,
count(*) as num_order_items,
sum(order_item_subtotal)
from order_items
group by order_item_order_id) x)
and order_amount > (select avg(order_amount) as avg_order_amount
from (select order_item_order_id as order_id,
count(*) as num_order_items,
sum(order_item_subtotal) as order_amount
from order_items
group by order_item_order_id) x)

```

```

with order_items_aggregated as (select order_item_order_id as order_id,
count(*) as num_order_items,
sum(order_item_subtotal) as order_total
from order_items
group by order_item_order_id),
order_items_avg as (select avg(num_order_items) as avg_order_items
from order_items_aggregated),
order_amnt_avg as (select avg(order_total) as avg_order_amount
from order_items_aggregated)
select order_id,
num_order_items,
order_total
from order_items_aggregated
where num_order_items > (select avg_order_items from order_items_avg)
and order_total > (select avg_order_amount from order_amnt_avg)

```

```

-- employees getting more than avg sal in their dept
select emp_id, name, salary, e.department_id, avg_salary
from employees e
inner join
(select department_id, avg(salary) as avg_salary
from employees
group by department_id) d
on e.department_id = d.department_id
where salary > avg_salary;

```

```

with dept_avg_salary as (select department_id, avg(salary) as avg_salary
from employees
group by department_id)
select emp_id, name, salary, e.department_id, avg_salary
from employees e
inner join dept_avg_salary d
on e.department_id = d.department_id
where salary > avg_salary;

```

```

select * from orders where order_id = 10000;
with cte_orders as (
select * from orders
)
select * from cte_orders where order_id = 10000;

```

if the above cte would have been materialized

50000 records are there in orders table

taking all these 50000 records and putting it in memory
cte_orders

it would actually require 2 full table scans in order to first materialize
the results and keep in memory and then to search for the given order id

Recursive CTE

=====

```
CREATE TABLE Employees (  
  EmployeeID INT PRIMARY KEY,  
  EmployeeName VARCHAR(100),  
  ManagerID INT  
);
```

```
INSERT INTO Employees (EmployeeID, EmployeeName, ManagerID) VALUES  
(1, 'John', NULL),      -- John is the CEO  
(2, 'Alice', 1),        -- Alice reports to John  
(3, 'Bob', 1),          -- Bob reports to John  
(4, 'Charlie', 2),      -- Charlie reports to Alice  
(5, 'David', 2),        -- David reports to Alice  
(6, 'Eva', 3);          -- Eva reports to Bob
```

EmployeeID | EmployeeName | ManagerID | Level

EmployeeID	EmployeeName	ManagerID	Level
1	John	NULL	0
2	Alice	1	1
3	Bob	1	1
4	Charlie	2	2
5	David	2	2
6	Eva	3	2

```
SELECT  
  EMPLOYEEID,  
  EMPLOYEENAME,  
  MANAGERID,  
  0 AS LEVEL  
FROM EMPLOYEES  
WHERE MANAGERID IS NULL;
```

```
SELECT  
  EMPLOYEEID,  
  EMPLOYEENAME,  
  MANAGERID,  
  1 AS LEVEL  
FROM EMPLOYEES  
WHERE MANAGERID = 1;
```

```

SELECT
EMPLOYEEID,
EMPLOYEEENAME,
MANAGERID,
2 AS LEVEL
FROM EMPLOYEES
WHERE MANAGERID = 2

```

UNION ALL

```

SELECT
EMPLOYEEID,
EMPLOYEEENAME,
MANAGERID,
2 AS LEVEL
FROM EMPLOYEES
WHERE MANAGERID = 3

```

when writing the recursive CTE there are 2 parts

1. the anchor member
2. recursive member

WITH EmployeeHierarchy as (

-- anchor member: select the top level employee (CEO)

```

select employeeid,
employeeename,
managerid,
0 as LEVEL
from employees
where managerid is null

```

union all

-- recursive member: find employees who report to the current employee

```

select
e.employeeid,
e.employeeename,
e.managerid,
eh.level + 1 -- increment the level for each recursion
from employees e
inner join EmployeeHierarchy eh
on e.managerid = eh.employeeid

```

)

```
select employeeid,  
employeeename,  
managerid,  
level  
from EmployeeHierarchy  
order by level, employeeid;
```

complete hierarchy for employee_id 6

```
empid. name. managerid
```

```
1      John   NULL
```

```
3      Bob    1
```

```
6      Eva    3
```

```
with EmployeeHierarchy as (
```

```
-- anchor member
```

```
select employeeid,  
employeeename,  
managerid  
from employees  
where employeeid = 6
```

```
-- recursive member
```

```
union all
```

```
select e.employeeid,  
e.employeeename,  
e.managerid  
from employees e  
inner join EmployeeHierarchy eh  
on e.employeeid = eh.managerid
```

```
)
```

```
select employeeid,  
employeeename,  
managerid  
from  
EmployeeHierarchy  
order by employeeid;
```

fibonacci series

=====

0 1 1 2 3 5 8 13 21

number = 0

next = 1

with fibonacci as (

-- anchor member

select 0 as number, 1 as next, 1 as level

union ALL

-- recursive member

select next, number + next, level + 1

from fibonacci

where level < 10

)

select number, level from fibonacci;